

Lab questions 202409

Question 4 - 1x

Objective

The primary objective of this question is to evaluate the student's comprehension of the security implications and potential exploitation techniques associated with SetUID/SetGID programs in Unix-like systems. It aims to assess their ability to identify vulnerabilities, understand the underlying mechanisms, and propose mitigation strategies.

– FROM HERE

Scenario

During a penetration test, you gain initial access to a Linux system as a low-privileged user.

While enumerating the system, you discover a SetUID binary named 'filemanager' owned by the 'root' user. The 'filemanager' binary is a command-line utility that allows users to navigate the file system, view, and edit files and their metadata.

You feel like you hit the jackpot.

Question

1. Describe three distinct ways an attacker could potentially exploit this SetUID binary to escalate their privileges to the 'root' user. For each way, provide a conceptual explanation and a practical example or technique.

– TO HERE

Answer

1. Three distinct ways an attacker could potentially exploit this SetUID binary:
 - a. **READ access to sensitive files:** Since 'filemanager' allows users to view and edit files, if it doesn't have proper access controls, an attacker could use it to read sensitive system files (e.g., /etc/shadow, configuration files) that they wouldn't normally have access to. This could lead to further privilege escalation (e.g., password bruteforcing via hash cracking) or information disclosure.
 - b. **WRITE access to sensitive files content:** 'filemanager' can also edit files. So we can modify sensitive system files (e.g., /etc/shadow, configuration files) to add a privileged user, change our permissions, change existing users' passwords, ...
 - c. **WRITE Access to files metadata:** this likely means we can arbitrarily change permissions and setuid/setgid bits of arbitrary files. If this is true, we can easily create a setuid shell.

Note: as discussed in class (slide #14 of "0x05 - Hacking Unix p1") here are a few important things to keep in mind, from a security perspective:

- Symlinks always have 0777 - Effective permissions are given by the target of the link
- Writing over/changing ownership on a setuid/setgid file removes setuid/setgid bits
- Most unices ignore the setuid attribute when applied to `#!/ scripts`
- Running a SetUID/SetGID program uses effective user/group ID (euid/egid)
 - These are different from uid/gid - some programs may drop privileges by default
- `LD_PRELOAD` and `LD_LIBRARY_PATH` are ignored for SetUID binaries

Question 5 - 1.5x

Objective

The objective of this question is to evaluate the student's ability to apply their knowledge of web application security, specifically in the context of JSON Web Tokens (JWTs) and Web exploitation.

The question also touches some concepts around encoding and integrity and authenticity measures a web application can use.

– FROM HERE

Scenario

You are a security consultant tasked with assessing the web application security of a client's e-commerce platform. During your reconnaissance phase, you discover that the application uses **JSON Web Tokens (JWTs)** for authentication and session management.

After creating an account, you get the following token:

None

```
eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjEyMzQ1Njc4OTAiLCJ1c2VybmFtZSI6Imd1cnJ5IiwiaWF0IjE1MTYyMzkwMjJ9.pSFNykK2aBr8ccZ2A7feUL7LihLZkWoDTFi1ThxBiB3Vaf45y9Mp71NdRf1HJKXAGdX0h1DSU8eoPWs-jmkX8fAn0NafXId84FIdUgE0vPIwVJ70sIZ6XXrZen-3IXfHN3RUZMoPSGD_jD1j4aLSwogLU_7TwFgqKkSra1v_b4Wt_-ePEpupntMJzdonw14-JWEYHKwxV9xBEaSw1Db90FqagkVQwmPMeTc13-C9RopW05bBTS9BDxFvJGxck0BAiEJ5TeUHFqboUvAG1IS1Ca491Wz-yJfgooxrF0bwxyzENu0Gekrz7lejCEPYUYpoJb-Ychi_6vLE0JzG2ZtRnQ
```

Once you decode it, you find its (simplified) content:

None

```
{
  "typ": "JWT"
  "alg": "RS256"
}
{
  "id": "1234567890",
  "username": "gerry",
  "admin": false
}
<signature>
```

The application seems quite outdated and you feel the JWT could be a viable candidate for subverting the application. But first, let's check our reasoning:

Questions

1. What encoding does JWT use? Describe the decoded JWT structure.
2. Where did you find the JWT? (i.e., where do you expect to find a JWT in general?) What is its purpose?
3. What type of signature the above JWT has?
4. Explain how you would attempt to exploit the JWT implementation/usage on the application you are pentesting, specifically targeting an administrative account. Detail the steps involved, including any tools or techniques you would employ.

– TO HERE

Answers

1. JWT is made of 3 parts, separated by a dot. Each part is base64 encoded:
 - a. **Header**: Contains metadata about the token, such as the algorithm used for signing ("alg": "RS256") and the token type ("typ": "JWT").
 - b. **Payload**: Contains the claims or statements, produced by the web application, about the user. Examples of claims are their ID ("id": "1234567890", for the case at hand), role ("admin": false, in our case).
 - c. **Signature**: A cryptographic hash of the header and payload, ensuring the token's integrity and authenticity. In this case, RS256 is used.
2. You found the JWT in the session cookie. It's used by the web application to assert the client's identity and privileges.
 - a. By verifying the signature of the JWT, the server can ensure that the token has not been tampered with and that the claims within it are trustworthy. This allows the server to confidently identify the client and grant them access to appropriate resources and functionalities based on the information embedded in the token.
3. This JWT uses RSA signature with SHA-256. That means it uses asymmetric encryption to sign the token (i.e., using the web application's private key), and it requires the web application's **public key** for verification.
4. The two ways we have mentioned in class (one of which was actually used to subvert the Juiceshop web app) are:
 - a. **Algorithm Downgrade**: The application might be vulnerable if it allows the alg header to be changed to 'none', effectively disabling signature verification and allowing for arbitrary modification of the payload.

- b. **Key Confusion:** If the server's **public key** is exposed (which it's not unlikely, given that it's a public key), an attacker could potentially change the alg header to HS256 and use the public key as a symmetric key to forge a valid signature. This approach would require looking for a symmetric key on the server, which could require some manual exploration or a URL enumeration via tools like Dirbuster or Gobuster. Specifically, I would also try to search for .PEM files.
- c. For both approaches, Burpsuite (for instance with the plugin we have seen in class) can help forge a new JWT and inject it. Alternatively, there is a plethora of open-source and web tools that can simplify the creation of a JWT.